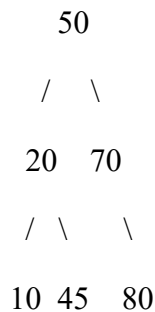
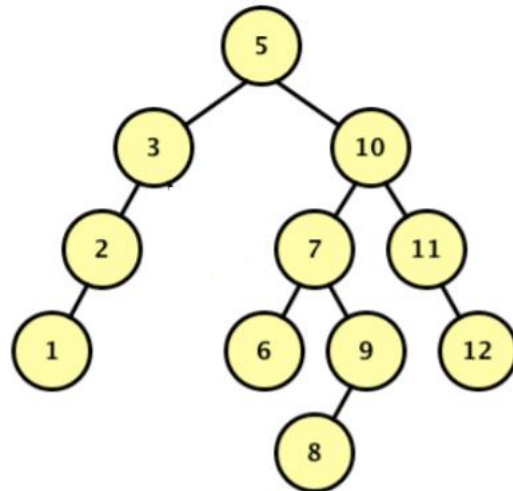


1. **a. Draw** the expression tree for $((a+(b-c))*(d-e))+(f*(h-i))$.
b. Find minimum and maximum numbers of nodes of a binary tree of height 6.
2. Suppose, you are given a string, S, and you have to find the length of the string. **Show** recursive definition of the problem by writing only the recursive function.
Example:
Input : Input string: one two three four five
Output : Length = 23
3. **Write** a program to convert an array into a Binary Search Tree by inserting the elements as nodes in BST, processing elements in the given order in the array. Also check whether the created Binary Search Tree is a near complete tree or not.
4. **Write** a program that will display a node of if the node has at least one child in a BST (Write only the Function). For example for the following BST, output will be 50, 20 and 70.



5. **Construct** an AVL tree using the data 23, 28, 32, 18, 21, 15, 38, 44, 33 and 58 entered as a sequential set. Clearly show the tree generated from each insertion. Also show the rotation that needs to be performed to balance the AVL tree.

6. Does the following BST is considered as an AVL tree? If it is not, then rebalance it to make an AVL tree. Next, delete the nodes containing 5 and 10 from the following tree. **Draw** each deleted scenario while you delete 5 and 10 from the below tree and make sure that, the tree created after each delete operation must be balanced.



7. A binary tree has eight nodes. The postorder and inorder traversals of the tree are given below. Draw the tree.

Postorder: FECHGDBA

Inorder: FCEABHDG

8. Ackerman's number, used in mathematical logic, can be calculated using the formula shown as following.

Ackerman (m, n)

$$= \begin{cases} n + 1 & \text{if } m = 0 \\ \text{Ackerman}(m - 1, 1) & \text{if } n = 0 \text{ and } m > 0 \\ \text{Ackerman}(m - 1, \text{Ackerman}(m, n - 1)) & \text{otherwise} \end{cases}$$

Write a recursive algorithm that calculates Ackerman's number. Verify your algorithm by using it to manually calculate the following test cases: Ackerman (2, 3) and Ackerman (3, 0).

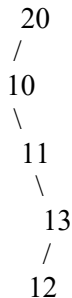
9. Given Preorder traversal of a BST, **write** a function to check if each non-leaf node has only one child. Assume that the BST contains unique entries.

Example:

Input: pre = {20, 10, 11, 13, 12}

Output: Yes

The given list represents following BST. In the following BST, every internal node has exactly 1 child. Therefore, the output is true.

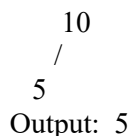


In Preorder traversal, descendants of every node appear after the node. In the above example, 20 is the first node in preorder and all descendants of 20 appear after it. All descendants of 20 are smaller than it. For 10, all descendants are greater than it. In general, we can say, if all internal nodes have only one child in a BST, then all the descendants of each node is either smaller or larger than the node. The reason is simple, since the tree is BST and every node has only one child, all descendants of a node will either be on left side or on right side means all descendants will either be smaller or greater.

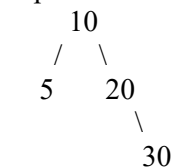
10. Given a Binary Search Tree (BST), **write** a function to find the second largest element.

Example:

Input:



Input:



Output: 20

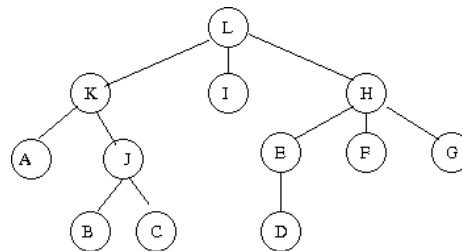
11. **Write** an algorithm that determines whether a binary tree is complete.

12. **Construct** an AVL tree using the following data entered as a sequential set. Show the balance factors in the resulting tree.

80 70 66 56 33 23 14 10 7

Next, delete the nodes containing 56 and 14 from the constructed tree. **Draw** each deleted scenario while you delete from the tree and make sure that, the tree created after each delete operation must be balanced.

13. **List** the nodes of the tree below in preorder and postorder.



14. Ackerman's number, used in mathematical logic, can be calculated using the formula shown as following.

Ackerman (m, n)

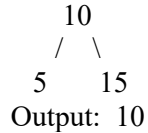
$$= \begin{cases} n + 1 & \text{if } m = 0 \\ \text{Ackerman}(m - 1, 1) & \text{if } n = 0 \text{ and } m > 0 \\ \text{Ackerman}(m - 1, \text{Ackerman}(m, n - 1)) & \text{otherwise} \end{cases}$$

Write a recursive algorithm that calculates Ackerman's number. Verify your algorithm by using it to manually calculate the following test cases: Ackerman (2, 5) and Ackerman (0, 3).

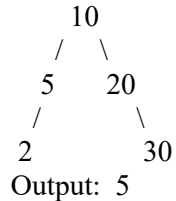
15. Given a Binary Search Tree (BST), **write** a function to find the second smallest element.

Example:

Input:



Input:



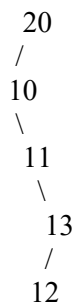
16. Given Preorder traversal of a BST, **write** a function to check if each non-leaf node has only one child. Assume that the BST contains unique entries.

Example:

Input: pre = {20, 10, 11, 13, 12}

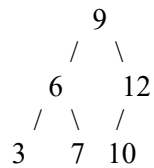
Output: Yes

The given list represents following BST. In the following BST, every internal node has exactly 1 child. Therefore, the output is true.



In Preorder traversal, descendants of every node appear after the node. In the above example, 20 is the first node in preorder and all descendants of 20 appear after it. All descendants of 20 are smaller than it. For 10, all descendants are greater than it. In general, we can say, if all internal nodes have only one child in a BST, then all the descendants of each node is either smaller or larger than the node. The reason is simple, since the tree is BST and every node has only one child, all descendants of a node will either be on left side or on right side means all descendants will either be smaller or greater.

17. Given a BST, **write** an algorithm to find the sum of all left leaves in it. For example, sum of all left leaves in below Binary Tree is $10+3=13$.



18. **Construct** an AVL tree using the following data entered as a sequential set. Show the balance factors in the resulting tree.

7 10 14 23 33 56 66 70 80

Next, delete the nodes containing 56 and 14 from the constructed tree. **Draw** each deleted scenario while you delete from the tree and make sure that, the tree created after each delete operation must be balanced.